IT HAPPENS @ RAALE #13

FleetWise

Vehicle & Delivery Assignment

LOGISTICS · STUDENT HANDOUT

Build a reliable fleet allocation system that assigns each delivery to an available vehicle and an available driver—without double-booking either resource.

Challenge	Build window	Core engineering rule
Fleet and delivery management	3 hours: 2 h 30 min build + 30 min report	Never assign an occupied vehicle or unavailable driver

Your mission: replace manual dispatch decisions with a clear, testable workflow. Create vehicles, drivers and deliveries; assign valid resources; update delivery progress; and show dispatchers what is happening now.

IT HAPPENS @ RAALE #13 · Build. Test. Explain.

PART 0 — What This Project Actually Is

0.1 The problem in plain language

A delivery company receives orders throughout the day. A dispatcher manually checks spreadsheets, calls drivers and tries to remember which vehicles are in use. Two deliveries can accidentally be assigned the same van, a driver may be allocated while already on a trip, and usable vehicles may sit idle because their availability is unclear.

FleetWise turns this process into one shared application. A dispatcher records fleet resources and delivery requests, assigns an eligible vehicle and driver, then tracks each delivery from creation to completion.

0.2 What success looks like

• A browser-based or clearly demonstrated application can manage vehicles, drivers and deliveries.

• The system shows availability and current assignment state, not just static records.

• Assignment validation blocks conflicts before they are saved.

• Delivery status changes are visible and follow a sensible lifecycle.

• A fresh run and a short test report prove that the rules work.

A form that saves assignments without checking conflicts is not FleetWise. The central challenge is enforcing business rules consistently—even when users click quickly or send duplicate requests.

0.3 Typical users

User	Main need
Dispatcher	Create deliveries and allocate resources safely.
Fleet manager	Maintain vehicle records and see utilisation.
Operations lead	Track delivery progress and identify blocked assignments.

PART 1 — Core Rules and Data Model

1.1 Required entities

Entity	Suggested fields	Rules / notes
Vehicle	vehicle_id, registration_no, type, capacity, status	Status: available, assigned, maintenance, inactive. Registration must be unique.
Driver	driver_id, name, phone, availability_status	Status: available, assigned, unavailable. Driver ID must be unique.
Delivery	delivery_id, pickup, dropoff, scheduled_time, priority, status	Status: pending, assigned, in_transit, delivered, cancelled.
Assignment	assignment_id, delivery_id, vehicle_id, driver_id, assigned_at	Store assignment history if time permits; prevent conflicting active assignments.

1.2 Mandatory assignment rule

A delivery can be assigned only when the selected vehicle and driver are both eligible. At minimum, an eligible vehicle is marked available and is not attached to another active delivery. The driver must be available and must not be attached to another active delivery.

```
IF delivery.status is pending
AND vehicle.status is available
AND driver.availability_status is available
AND no active assignment uses vehicle
AND no active assignment uses driver:
    create assignment
    mark vehicle assigned
    mark driver assigned
    mark delivery assigned
ELSE:
    reject assignment with clear reason
```

1.3 Define “active” clearly

For this challenge, assigned and in_transit deliveries count as active. Delivered and cancelled deliveries do not block future assignments. Keep this rule consistent across the UI, API and database.

Do not rely only on a disabled button in the user interface. Validate again on the server when an assignment is submitted.

PART 2 — User Workflows

2.1 Vehicle management

• Add vehicle with unique registration number, vehicle type and capacity.

• View all vehicles with current status and active delivery, if any.

• Change vehicle to maintenance or inactive only when no active assignment exists; otherwise reject or require the assignment to be resolved first.

• Allow a released vehicle to become available again after delivery completion or cancellation.

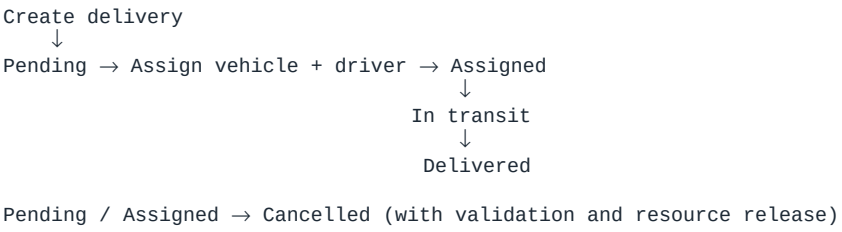
2.2 Driver management

• Add driver with unique ID, name and contact information.

• Display driver availability and assigned delivery.

• Do not mark a driver unavailable while the driver has an active delivery unless your workflow first safely ends or reassigns that delivery.

2.3 Delivery workflow



2.4 Assignment flow

Dispatcher opens a pending delivery, selects an available vehicle and driver, and submits. FleetWise validates all rules, saves the assignment, updates statuses and shows confirmation. If either resource is occupied, the request must fail without partial updates.

The application must never leave a vehicle marked assigned while the delivery remains pending because a later database write failed. Use a transaction or an equivalent safe update strategy.

PART 3 — Suggested Architecture

Use a simple architecture that your team can finish and explain within the event time. Any programming language and database are acceptable if your organisers permit them.

Layer	Responsibility	Practical choice
UI	Vehicle, driver and delivery forms; assignment screen; status board	HTML/CSS/JavaScript or React
API / business logic	Validation, assignment, status transitions, error responses	Python FastAPI/Flask or Node.js/Express
Database	Persist entities, statuses and assignments	SQLite for speed; PostgreSQL/MySQL if ready
Tests	Prove conflict prevention and status correctness	Unit tests plus API/UI checks

3.1 Logical components

```
Dispatcher UI
   |  create / list / assign / update status
   v
FleetWise API
   |-- validate vehicle eligibility
   |-- validate driver eligibility
   |-- validate delivery state
   |-- commit assignment atomically
   v
Database
   |-- Vehicles
   |-- Drivers
   |-- Deliveries
   `-- Assignments
```

3.2 Important implementation notes

• Use database transactions for assignment creation and resource-status updates.

• Where supported, enforce uniqueness or active-assignment constraints at database level too.

• Handle two simultaneous assignment requests safely; both must not reserve the same vehicle or driver.

• Return useful error messages such as “Vehicle V-102 is already assigned to an active delivery.”

• Keep credentials and configuration outside source code if using a hosted database.

Do not spend most of the build window polishing the dashboard. First prove that valid assignments succeed and conflicting assignments fail.

PART 4 — API Contract (Suggested)

You may adapt endpoint names, but document your actual routes and demonstrate equivalent functionality.

Method / endpoint	Purpose	Expected behaviour
POST /vehicles	Create vehicle	Reject duplicate registration; validate fields.
GET /vehicles	List vehicles	Include availability / current status.
POST /drivers	Create driver	Reject duplicate driver ID; validate fields.
GET /drivers	List drivers	Include availability / current status.
POST /deliveries	Create delivery	Create delivery in pending state.
GET /deliveries	List deliveries	Show status and assignment details.
POST /deliveries/{id}/assign	Assign vehicle + driver	Validate all constraints; no partial update on failure.
PATCH /deliveries/{id}/status	Update status	Allow only valid transitions; release resources when appropriate.
GET /health	Health check	Return service status; optional but recommended.

4.1 Example assignment request

```
POST /deliveries/D-204/assign
{
  "vehicle_id": "V-102",
  "driver_id": "DR-018"
}
```

4.2 Example success response

```
{
  "delivery_id": "D-204",
  "vehicle_id": "V-102",
  "driver_id": "DR-018",
  "status": "assigned",
  "message": "Assignment created successfully"
}
```

4.3 Example conflict response

```
{
  "error": "VEHICLE_UNAVAILABLE",
  "message": "Vehicle V-102 is already assigned to an active delivery"
}
```

Use suitable HTTP status codes (for example, 201 for creation, 200 for success, 400 for invalid input, 404 for missing records and 409 for assignment conflicts).

PART 5 — Testing and Edge Cases

A working demo is not enough. Test the business rules deliberately, including failures and repeated requests.

Test	Action	Expected result
T1 — Valid assignment	Assign available vehicle and driver to pending delivery.	Success; all three records reflect assignment.
T2 — Vehicle conflict	Assign same vehicle to second active delivery.	Rejected; no records partially changed.
T3 — Driver conflict	Assign same driver to second active delivery.	Rejected; no records partially changed.
T4 — Both occupied	Choose occupied vehicle and occupied driver.	Rejected with clear reason.
T5 — Unavailable driver	Set driver unavailable, then assign.	Rejected.
T6 — Maintenance vehicle	Select vehicle in maintenance.	Rejected.
T7 — Unknown ID	Use missing vehicle, driver or delivery ID.	Rejected safely; no crash.
T8 — Complete delivery	Mark assigned/in-transit delivery delivered.	Delivery closes; vehicle and driver become available.
T9 — Cancel delivery	Cancel eligible delivery.	Status changes; resources released consistently.
T10 — Invalid transition	Mark pending delivery delivered directly.	Rejected or handled by documented workflow.
T11 — Duplicate submit	Submit same assignment twice quickly.	No duplicate or inconsistent active assignment.
T12 — Concurrent request	Two requests attempt same vehicle at once.	At most one succeeds.

For each test, record input, observed output, pass/fail and evidence (screenshot, API response or test log). Never report a test as passed unless you ran it.

PART 6 — Build Plan and Must-Haves

6.1 Three-hour timeline

Time	Phase	Checkpoint
5:00–5:10	Read, agree rules, split work	Everyone knows data model and task owner.
5:10–5:35	Skeleton	App starts; records can be created or seeded.
5:35–6:05	CRUD workflows	Vehicle, driver and delivery lists/forms work.
6:05–6:40	Assignment logic	Valid assignment works; conflicts are blocked.
6:40–7:00	Status lifecycle	Assigned → in transit → delivered; release resources.
7:00–7:20	Test and harden	Run conflict and edge-case tests.
7:20–7:30	Build freeze	Commit, verify clean start, stop feature work.
7:30–8:00	Report and demo prep	Complete REPORT.md and prepare evidence.

6.2 Must-have checklist

- Manage vehicles, drivers and deliveries.
- Assign a vehicle and driver to a pending delivery.
- Block occupied vehicles and unavailable/occupied drivers.
- Prevent partial updates when assignment fails.
- Track delivery status and release resources at the correct point.
- Validate missing IDs and invalid status transitions.
- Provide clear user-facing errors.
- Include reproducible test evidence and a run guide.

Build the end-to-end skeleton first. Add features only after the assignment rule is proven.

PART 7 — Stretch Goals and Marking

7.1 Stretch goals

• Vehicle capacity or type matching against delivery requirements.

• Priority-based delivery queue and basic dispatch recommendations.

• Search/filter by status, driver, vehicle or delivery ID.

• Audit log showing who assigned or changed each delivery.

• Dashboard metrics: available fleet, active deliveries, delivered count and utilisation.

• Map or route display, if time and approved services permit.

• Role-based access for dispatcher and fleet manager.

• Concurrent-request test demonstrating safe allocation.

7.2 Marking rubric — 100 marks

Component	Marks
Vehicle, driver and delivery management	15
Assignment logic and conflict prevention	25
Status lifecycle and resource release	15
Persistence, validation and error handling	10
Usability and operational visibility	10
Testing: edge cases and evidence	15
Report, design decisions and reproducibility	10
Total	100

The core rule carries the highest weight. A beautiful interface that permits double-booking cannot pass as a reliable dispatch system.

7.3 Demo acceptance

Judges should see one successful assignment, then attempt to reuse the same vehicle, then reuse the same driver. Both conflict attempts must be rejected. Finally, complete the first delivery and demonstrate that resources become available again.

PART 8 — The Report (30 minutes)

Submit one file: REPORT.md, or equivalent clearly structured report. Keep it factual and concise.

8.1 Required sections

- **What we built:** explain the application, architecture and working features.
- **Data model:** list entities, key fields, relationships and status values.
- **Assignment rule:** explain how vehicle and driver conflicts are prevented, including concurrent requests.
- **Methods and decisions:** chosen stack, database, validation approach and rejected alternatives.
- **Results:** test table with at least eight executed cases, expected result, observed result and pass/fail.
- **Team work:** owner for each component, planned versus actual progress and one dead end.
- **Limitations and next steps:** state concrete weaknesses and what would fix them.
- **How to run:** exact install/start commands, environment setup and demo credentials if applicable.

8.2 Final pre-submission checklist

- Application starts from documented steps.
- Vehicle and driver conflict tests pass.
- Failed assignment leaves database state unchanged.
- Delivery status transitions are validated.
- Completed/cancelled deliveries release resources consistently.
- Test evidence matches report claims.
- Code and report are committed or submitted as instructed.

Final principle: FleetWise must make dispatch safer, not merely faster. Show evidence that the system cannot allocate the same vehicle or driver to overlapping active deliveries.

Good luck — keep assignments safe, status clear, and evidence honest.